**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Институт компьютерных наук
Кафедра автоматизированных систем управления

# Лабораторная работа №4

по дисциплине «Машинное обучение»

Студент М-РИТ-25-1 _____ Станиславчук С. М.

(подпись, дата)

Руководитель

Доцент, д.т.н. _____ Сараев П. В.

(подпись, дата)

Липецк 2025

1. Задание кафедры

1) Загрузите из UCI ML Repository базу данных Energy efficiency.
http://archive.ics.uci.edu/ml/datasets/Energy+efficiency
В этой базе представлены результаты исследования энергоэффективности зданий в виде отопительной (heating) и холодильной (cooling) нагрузки в зависимости от параметров

2) Разделите данные на обучающее (80%) и тестовое(20%) множество

3) Постройте различные математические и нейросетевые модели выходов y1 и y2 с использованием функций lm, glm, nlm и функций выбранного пакета для нейросетевого моделирования

4) Оцените качество постронных моделей на тестовом множестве. Постройте графиик, иллюстрирующие качество моделирования.

5) Сведите результаты качества построения моделей в таблицу. Выберите лучшую модель.

6) Для лучшей по точности на тестовом множестве линейной по параметрам модели определите значения параметров в разных нормах, в том числе для ошибки MAPE с применением медианной регрессии

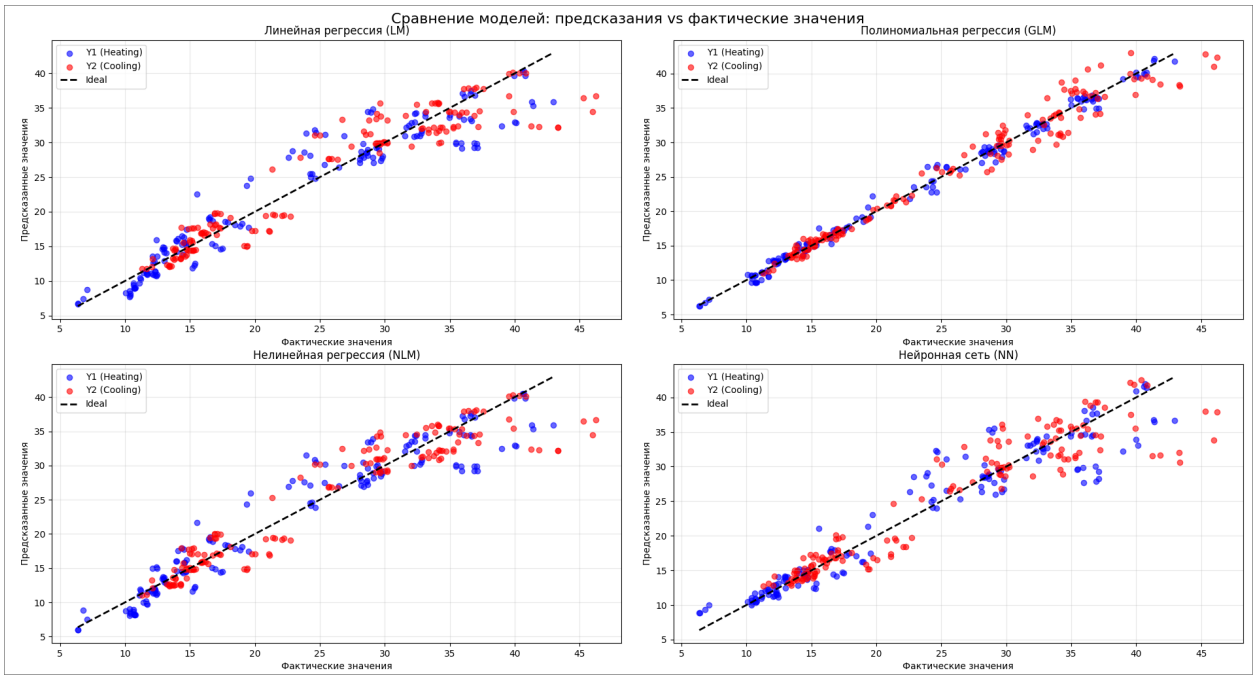
2. Цель работы

Цель работы — получение навыков пострения математических моделей средствами языка R (или Python) с использованием разных норм для определения величин ошибок.

3. Ход работы

В качестве языка программирования для выполнения лабораторной работы, я выбрал Python.

Пример выполнения программы:




Из рисунка видно, что GLM отработала лучше всего, результаты для обеих Y расположены плотнее всего к идеальной диагонали. Остальные модели показывают схожий результат.

```
Linear Regression (lm)
Y1 (Heating) - R²: 0.9122, RMSE: 3.0254
Y2 (Cooling) - R²: 0.8932, RMSE: 3.1454

Polynomial Regression (glm)
Y1 (Heating) - R²: 0.9938, RMSE: 0.8030
Y2 (Cooling) - R²: 0.9678, RMSE: 1.7260
Nonlinear Model (nlm) for Y1 - complex
Y1 (Heating) - R²: 0.9167, RMSE: 2.9467

Nonlinear Model (nlm) for Y2 - complex
Y2 (Cooling) - R²: 0.8975, RMSE: 3.0824

Neural Network (nnet)
Y1 (Heating) - R²: 0.9177, RMSE: 2.9290
Y2 (Cooling) - R²: 0.8887, RMSE: 3.2111

Model Comparison Results:
  Model     R²_Y1    RMSE_Y1     R²_Y2    RMSE_Y2
0    lm  0.912184   3.025424  0.893226   3.145382
1   glm  0.993814   0.802956  0.967849   1.725987
2   nlm  0.916696   2.946678  0.897456   3.082445
3  nnet  0.917693   2.928996  0.888716   3.211109

Лучшая модель для Y1 (Heating Load): glm (R² = 0.9938)
Лучшая модель для Y2 (Cooling Load): glm (R² = 0.9678)

Detailed Results:
  Model Target        R²      RMSE
0    lm     Y1  0.912184  3.025424
1    lm     Y2  0.893226  3.145382
2   glm     Y1  0.993814  0.802956
3   glm     Y2  0.967849  1.725987
4   nlm     Y1  0.916696  2.946678
5   nlm     Y2  0.897456  3.082445
6  nnet     Y1  0.917693  2.928996
7  nnet     Y2  0.888716  3.211109
```

Из таблицы видно, что наименьшей ошибкой RMSE для Y1 и Y2 обладает модель GLM (0.8 и 1.72).

Анализ параметров лучшей модели (GLM) в разных нормах:

```
Количество признаков после полиномиального преобразования: 44
Первые 10 признаков: ['X1' 'X2' 'X3' 'X4' 'X5' 'X6' 'X7' 'X8' 'X1^2' 'X1 X2']
6.1 МНК РЕГРЕССИЯ (L2 НОРМА)

Параметры для Y1 (Heating Load):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -1939.893734      1939.893734
8     X1^2 -1897.221691      1897.221691
11   X1 X4 -1607.889697      1607.889697
30   X4 X5   993.833620       993.833620
19   X2 X5   873.310770       873.310770
4       X5   630.917189       630.917189
10   X1 X3  -610.143980       610.143980
17   X2 X3  -489.802281       489.802281
24   X3 X4  -487.946345       487.946345
16    X2^2  -396.794312       396.794312

Intercept для Y1: 460.482441

Параметры для Y2 (Cooling Load):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -2740.207621      2740.207621
8     X1^2 -2684.190042      2684.190042
11   X1 X4 -2265.981653      2265.981653
30   X4 X5  1398.908160      1398.908160
19   X2 X5  1221.571930      1221.571930
4       X5   899.052983       899.052983
10   X1 X3  -872.759602       872.759602
17   X2 X3  -701.777697       701.777697
24   X3 X4  -698.472270       698.472270
16    X2^2  -553.164519       553.164519

Intercept для Y2: 624.616188
```

```
6.2 MAE РЕГРЕССИЯ (L1 НОРМА) - МЕДИАННАЯ РЕГРЕССИЯ

Параметры для Y1 (L1 норма):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -1937.560935      1937.560935
8     X1^2 -1897.329428      1897.329428
11   X1 X4 -1605.052632      1605.052632
30   X4 X5   914.031782       914.031782
19   X2 X5   808.833674       808.833674
4       X5   769.135582       769.135582
10   X1 X3  -611.285099       611.285099
17   X2 X3  -490.199922       490.199922
24   X3 X4  -488.137249       488.137249
3       X4   469.115246       469.115246

Параметры для Y2 (L1 норма):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -2737.084909      2737.084909
8     X1^2 -2684.336736      2684.336736
11   X1 X4 -2262.512315      2262.512315
30   X4 X5  1290.731779      1290.731779
19   X2 X5  1134.332833      1134.332833
4       X5  1086.564835      1086.564835
10   X1 X3  -873.604381       873.604381
17   X2 X3  -702.070840       702.070840
24   X3 X4  -698.525506       698.525506
3       X4   645.546247       645.546247
```

```
Параметры для Y1 (MAPE):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -1937.505566      1937.505566
8     X1^2 -1897.276101      1897.276101
11   X1 X4 -1605.146851      1605.146851
30   X4 X5   914.114583       914.114583
19   X2 X5   808.887635       808.887635
4       X5   769.280872       769.280872
10   X1 X3  -610.976805       610.976805
17   X2 X3  -490.106863       490.106863
24   X3 X4  -488.072173       488.072173
3       X4   469.206671       469.206671

Параметры для Y2 (MAPE):
   Feature  Coefficient  Abs_Coefficient
9    X1 X2 -2736.983999      2736.983999
8     X1^2 -2684.377642      2684.377642
11   X1 X4 -2262.188812      2262.188812
30   X4 X5  1290.574641      1290.574641
19   X2 X5  1133.935978      1133.935978
4       X5  1086.617872      1086.617872
10   X1 X3  -874.071682       874.071682
17   X2 X3  -702.326360       702.326360
24   X3 X4  -698.780856       698.780856
3       X4   645.623440       645.623440
```

```
6.4 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ РАЗНЫХ НОРМ
Сравнение производительности:
              Model Target     MSE    RMSE     MAE    MAPE
0         L2 (МНК)     Y1  0.6447  0.8030  0.6042  3.0542
1  L1 (Медианная)     Y1  0.6652  0.8156  0.5970  3.0150
2         L2 (МНК)     Y2  2.9790  1.7260  1.1909  4.2014
3  L1 (Медианная)     Y2  3.3749  1.8371  1.1925  4.1870
```

```
6.5 АНАЛИЗ НАИБОЛЕЕ ВАЖНЫХ ПРИЗНАКОВ
Топ-10 признаков для Y1 по разным нормам:
  L2_Norm L1_Norm
0   X1 X2   X1 X2
1    X1^2    X1^2
2   X1 X4   X1 X4
3   X4 X5   X4 X5
4   X2 X5   X2 X5
5      X5      X5
6   X1 X3   X1 X3
7   X2 X3   X2 X3
8   X3 X4   X3 X4
9    X2^2      X4

Лучшая модель по MAPE для Y1: L1 (Медианная) (MAPE = 3.01%)
Лучшая модель по MAPE для Y2: L1 (Медианная) (MAPE = 4.19%)

Топ-5 наиболее важных признаков для энергоэффективности:
Y1 (Heating Load):
  1. X1 X2
  2. X1^2
  3. X1 X4
  4. X4 X5
  5. X2 X5

Y2 (Cooling Load):
  1. X1 X2
  2. X1^2
  3. X1 X4
  4. X4 X5
  5. X2 X5
```

3. Вывод

GLM отработала лучше всего, результаты для обеих Y расположены плотнее всего к идеальной диагонали. Остальные модели показывают схожий результат.

Для RMSE/MSE лучше работает L2 (МНК), для MAPE L1 (медианная регрессия).

Итоговые результаты однозначно показывают, что полиномиальная регрессия является оптимальной моделью для данного набора данных.

# Приложение 1

```
# 1. Загрузите из UCI ML Repository базу данных Energy efficiency.
# http://archive.ics.uci.edu/ml/datasets/Energy+efficiency
# В этой базе представлены результаты исследования энергоэффективности
# зданий в виде отопительной (heating) и холодильной (cooling) нагрузки
# в зависимости от параметров

import pandas as pd
import numpy as np

df = pd.read_csv("energy_efficiency.csv")

X = df.iloc[:, :-2]  # first 8 columns (features)
y = df.iloc[:, -2:]  # last 2 columns (targets)

print("Dataset loaded locally!")
print(X.head())
print(y.head())


# 2. Разделите данные на обучающее (80%) и тестовое(20%) мнодество
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

# 3. Постройте различные математические и нейросетевые модели выходов
# y1 и y2 с использованием функций lm, glm, nlm и функций выбранного пакета для
# нейросетевого моделирования
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

from sklearn.metrics import r2_score, mean_squared_error

def eval_model_separate(name, y_true, y_pred):
    """Оценка модели отдельно для Y1 и Y2"""
    print(f"\n{name}")

    # Для Y1
    r2_y1 = r2_score(y_true.iloc[:, 0], y_pred[:, 0])
    rmse_y1 = np.sqrt(mean_squared_error(y_true.iloc[:, 0], y_pred[:, 0]))
    print(f"Y1 (Heating) - R²: {r2_y1:.4f}, RMSE: {rmse_y1:.4f}")

    # Для Y2
    r2_y2 = r2_score(y_true.iloc[:, 1], y_pred[:, 1])
    rmse_y2 = np.sqrt(mean_squared_error(y_true.iloc[:, 1], y_pred[:, 1]))
    print(f"Y2 (Cooling) - R²: {r2_y2:.4f}, RMSE: {rmse_y2:.4f}")

# lm
from sklearn.linear_model import LinearRegression
lm_model = LinearRegression()
lm_model.fit(X_train_scaled, y_train)
```

```
pred_lm = lm_model.predict(X_test_scaled)
eval_model_separate("Linear Regression (lm)", y_test, pred_lm)

# glm
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

glm_model = LinearRegression()
glm_model.fit(X_train_poly, y_train)
pred_glm = glm_model.predict(X_test_poly)
eval_model_separate("Polynomial Regression (glm)", y_test, pred_glm)

# nlm
from scipy.optimize import curve_fit

from sklearn.preprocessing import MinMaxScaler
scaler_minmax = MinMaxScaler()
X_train_minmax = scaler_minmax.fit_transform(X_train)
X_test_minmax = scaler_minmax.transform(X_test)

def nonlinear_func(X, a, b, c, d, e, f, g, h, i):
    # Линейная комбинация + нелинейные члены
    result = (a * X[:, 0] + b * X[:, 1] + c * X[:, 2] + d * X[:, 3] +
              e * X[:, 4] + f * X[:, 5] + g * X[:, 6] + h * X[:, 7] +
              i * (X[:, 0] * X[:, 1]))
    return result

def simple_nonlinear_func(X, a, b, c, d):
    return a + b * X[:, 0] + c * X[:, 1] + d * (X[:, 0] * X[:, 1])

def debug_nonlinear_func(X, a, b):
    return a * X[:, 0] + b * X[:, 1]

# Для Y1
try:
    # Начальные приближения параметров
    initial_guess = [1.0] * 9  # 9 параметров для nonlinear_func
    popt_y1, pcov_y1 = curve_fit(nonlinear_func, X_train_minmax, y_train["Y1"],
                                 p0=initial_guess, maxfev=10000)
    pred_nlm_y1 = nonlinear_func(X_test_minmax, *popt_y1)
    print("Nonlinear Model (nlm) for Y1 - complex")
    r2_y1 = r2_score(y_test["Y1"], pred_nlm_y1)
    rmse_y1 = np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1))
    print(f"Y1 (Heating) - R²: {r2_y1:.4f}, RMSE: {rmse_y1:.4f}")
except Exception as e:
    print(f"Complex NLM for Y1 failed: {e}")
    try:
        # Пробуем упрощенную функцию
        initial_guess_simple = [1.0] * 4
        popt_y1, pcov_y1 = curve_fit(simple_nonlinear_func, X_train_minmax, y_train["Y1"],
                                     p0=initial_guess_simple, maxfev=5000)
        pred_nlm_y1 = simple_nonlinear_func(X_test_minmax, *popt_y1)
        print("Nonlinear Model (nlm) for Y1 - simplified")
        r2_y1 = r2_score(y_test["Y1"], pred_nlm_y1)
        rmse_y1 = np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1))
        print(f"Y1 (Heating) - R²: {r2_y1:.4f}, RMSE: {rmse_y1:.4f}")
    except Exception as e2:
        print(f"Simplified NLM for Y1 failed: {e2}")
        # Самый простой вариант
        try:
            initial_guess_debug = [1.0, 1.0]
            popt_y1, pcov_y1 = curve_fit(debug_nonlinear_func, X_train_minmax, y_train["Y1"],
                                         p0=initial_guess_debug, maxfev=1000)
            pred_nlm_y1 = debug_nonlinear_func(X_test_minmax, *popt_y1)
            print("Nonlinear Model (nlm) for Y1 - debug version")
            r2_y1 = r2_score(y_test["Y1"], pred_nlm_y1)
            rmse_y1 = np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1))
            print(f"Y1 (Heating) - R²: {r2_y1:.4f}, RMSE: {rmse_y1:.4f}")
```

```
        except Exception as e3:
            print(f"Debug NLM for Y1 failed: {e3}")
            # Резервный вариант - используем среднее значение
            pred_nlm_y1 = np.full_like(y_test["Y1"], y_train["Y1"].mean())
            print("Nonlinear Model (nlm) for Y1 - using mean as fallback")
            r2_y1 = r2_score(y_test["Y1"], pred_nlm_y1)
            rmse_y1 = np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1))
            print(f"Y1 (Heating) - R²: {r2_y1:.4f}, RMSE: {rmse_y1:.4f}")

# Для Y2
try:
    initial_guess = [1.0] * 9
    popt_y2, pcov_y2 = curve_fit(nonlinear_func, X_train_minmax, y_train["Y2"],
                                 p0=initial_guess, maxfev=10000)
    pred_nlm_y2 = nonlinear_func(X_test_minmax, *popt_y2)
    print("\nNonlinear Model (nlm) for Y2 - complex")
    r2_y2 = r2_score(y_test["Y2"], pred_nlm_y2)
    rmse_y2 = np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2))
    print(f"Y2 (Cooling) - R²: {r2_y2:.4f}, RMSE: {rmse_y2:.4f}")
except Exception as e:
    print(f"Complex NLM for Y2 failed: {e}")
    try:
        initial_guess_simple = [1.0] * 4
        popt_y2, pcov_y2 = curve_fit(simple_nonlinear_func, X_train_minmax, y_train["Y2"],
                                     p0=initial_guess_simple, maxfev=5000)
        pred_nlm_y2 = simple_nonlinear_func(X_test_minmax, *popt_y2)
        print("Nonlinear Model (nlm) for Y2 - simplified")
        r2_y2 = r2_score(y_test["Y2"], pred_nlm_y2)
        rmse_y2 = np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2))
        print(f"Y2 (Cooling) - R²: {r2_y2:.4f}, RMSE: {rmse_y2:.4f}")
    except Exception as e2:
        print(f"Simplified NLM for Y2 failed: {e2}")
        try:
            initial_guess_debug = [1.0, 1.0]
            popt_y2, pcov_y2 = curve_fit(debug_nonlinear_func, X_train_minmax, y_train["Y2"],
                                         p0=initial_guess_debug, maxfev=1000)
            pred_nlm_y2 = debug_nonlinear_func(X_test_minmax, *popt_y2)
            print("Nonlinear Model (nlm) for Y2 - debug version")
            r2_y2 = r2_score(y_test["Y2"], pred_nlm_y2)
            rmse_y2 = np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2))
            print(f"Y2 (Cooling) - R²: {r2_y2:.4f}, RMSE: {rmse_y2:.4f}")
        except Exception as e3:
            print(f"Debug NLM for Y2 failed: {e3}")
            pred_nlm_y2 = np.full_like(y_test["Y2"], y_train["Y2"].mean())
            print("Nonlinear Model (nlm) for Y2 - using mean as fallback")
            r2_y2 = r2_score(y_test["Y2"], pred_nlm_y2)
            rmse_y2 = np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2))
            print(f"Y2 (Cooling) - R²: {r2_y2:.4f}, RMSE: {rmse_y2:.4f}")

pred_nlm_combined = np.column_stack([pred_nlm_y1, pred_nlm_y2])

# nn
nn = MLPRegressor(hidden_layer_sizes=(16, 8), activation='relu',
                  solver='adam', max_iter=2000, random_state=42)
nn.fit(X_train_scaled, y_train)
pred_nn = nn.predict(X_test_scaled)
eval_model_separate("Neural Network (nnet)", y_test, pred_nn)


# 4. Оцените качество постронных моделей на тестовом множестве.
# Постройте графиик, иллюстрирующие качество моделирования.

import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Сравнение моделей: предсказания vs фактические значения', fontsize=16)

# График 1: Линейная регрессия (LM)
ax1 = axes[0, 0]
ax1.scatter(y_test["Y1"], pred_lm[:, 0], alpha=0.6, color='blue', label='Y1 (Heating)')
```

```
ax1.scatter(y_test["Y2"], pred_lm[:, 1], alpha=0.6, color='red', label='Y2 (Cooling)')
ax1.plot([y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         [y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         'k--', lw=2, label='Ideal')
ax1.set_xlabel("Фактические значения")
ax1.set_ylabel("Предсказанные значения")
ax1.set_title("Линейная регрессия (LM)")
ax1.legend()
ax1.grid(True, alpha=0.3)

# График 2: Полиномиальная регрессия (GLM)
ax2 = axes[0, 1]
ax2.scatter(y_test["Y1"], pred_glm[:, 0], alpha=0.6, color='blue', label='Y1 (Heating)')
ax2.scatter(y_test["Y2"], pred_glm[:, 1], alpha=0.6, color='red', label='Y2 (Cooling)')
ax2.plot([y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         [y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         'k--', lw=2, label='Ideal')
ax2.set_xlabel("Фактические значения")
ax2.set_ylabel("Предсказанные значения")
ax2.set_title("Полиномиальная регрессия (GLM)")
ax2.legend()
ax2.grid(True, alpha=0.3)

# График 3: Нелинейная регрессия (NLM)
ax3 = axes[1, 0]
ax3.scatter(y_test["Y1"], pred_nlm_y1, alpha=0.6, color='blue', label='Y1 (Heating)')
ax3.scatter(y_test["Y2"], pred_nlm_y2, alpha=0.6, color='red', label='Y2 (Cooling)')
ax3.plot([y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         [y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         'k--', lw=2, label='Ideal')
ax3.set_xlabel("Фактические значения")
ax3.set_ylabel("Предсказанные значения")
ax3.set_title("Нелинейная регрессия (NLM)")
ax3.legend()
ax3.grid(True, alpha=0.3)

# График 4: Нейронная сеть (NN)
ax4 = axes[1, 1]
ax4.scatter(y_test["Y1"], pred_nn[:, 0], alpha=0.6, color='blue', label='Y1 (Heating)')
ax4.scatter(y_test["Y2"], pred_nn[:, 1], alpha=0.6, color='red', label='Y2 (Cooling)')
ax4.plot([y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         [y_test.iloc[:, 0].min(), y_test.iloc[:, 0].max()],
         'k--', lw=2, label='Ideal')
ax4.set_xlabel("Фактические значения")
ax4.set_ylabel("Предсказанные значения")
ax4.set_title("Нейронная сеть (NN)")
ax4.legend()
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Дополнительный график: сравнение всех моделей для Y1 и Y2 отдельно
fig2, (ax5, ax6) = plt.subplots(1, 2, figsize=(15, 6))

# Y1 сравнение
ax5.scatter(y_test["Y1"], pred_lm[:, 0], alpha=0.6, color='orange', label='LM')
ax5.scatter(y_test["Y1"], pred_glm[:, 0], alpha=0.6, color='green', label='GLM')
ax5.scatter(y_test["Y1"], pred_nlm_y1, alpha=0.6, color='purple', label='NLM')
ax5.scatter(y_test["Y1"], pred_nn[:, 0], alpha=0.6, color='brown', label='NN')
ax5.plot([y_test["Y1"].min(), y_test["Y1"].max()],
         [y_test["Y1"].min(), y_test["Y1"].max()],
         'k--', lw=2, label='Ideal')
ax5.set_xlabel("Фактические значения Y1")
ax5.set_ylabel("Предсказанные значения Y1")
ax5.set_title("Сравнение всех моделей для Heating Load (Y1)")
ax5.legend()
ax5.grid(True, alpha=0.3)

# Y2 сравнение
```

```
ax6.scatter(y_test["Y2"], pred_lm[:, 1], alpha=0.6, color='orange', label='LM')
ax6.scatter(y_test["Y2"], pred_glm[:, 1], alpha=0.6, color='green', label='GLM')
ax6.scatter(y_test["Y2"], pred_nlm_y2, alpha=0.6, color='purple', label='NLM')
ax6.scatter(y_test["Y2"], pred_nn[:, 1], alpha=0.6, color='brown', label='NN')
ax6.plot([y_test["Y2"].min(), y_test["Y2"].max()],
         [y_test["Y2"].min(), y_test["Y2"].max()],
         'k--', lw=2, label='Ideal')
ax6.set_xlabel("Фактические значения Y2")
ax6.set_ylabel("Предсказанные значения Y2")
ax6.set_title("Сравнение всех моделей для Cooling Load (Y2)")
ax6.legend()
ax6.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 5. Сведите результаты качества построения моделей в таблицу. Выберите
# лучшую модель.
import pandas as pd

results = {
    "Model": ["lm", "glm", "nlm", "nnet"],
    "R²_Y1": [
        r2_score(y_test["Y1"], pred_lm[:, 0]),
        r2_score(y_test["Y1"], pred_glm[:, 0]),
        r2_score(y_test["Y1"], pred_nlm_y1),
        r2_score(y_test["Y1"], pred_nn[:, 0])
    ],
    "RMSE_Y1": [
        np.sqrt(mean_squared_error(y_test["Y1"], pred_lm[:, 0])),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_glm[:, 0])),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1)),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_nn[:, 0]))
    ],
    "R²_Y2": [
        r2_score(y_test["Y2"], pred_lm[:, 1]),
        r2_score(y_test["Y2"], pred_glm[:, 1]),
        r2_score(y_test["Y2"], pred_nlm_y2),
        r2_score(y_test["Y2"], pred_nn[:, 1])
    ],
    "RMSE_Y2": [
        np.sqrt(mean_squared_error(y_test["Y2"], pred_lm[:, 1])),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_glm[:, 1])),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2)),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_nn[:, 1]))
    ]
}

results_df = pd.DataFrame(results)
print("\nModel Comparison Results:")
print(results_df)

best_y1_idx = results_df["R²_Y1"].idxmax()
best_y2_idx = results_df["R²_Y2"].idxmax()

print(f"\nЛучшая модель для Y1 (Heating Load): {results_df.loc[best_y1_idx, 'Model']} (R² =
{results_df.loc[best_y1_idx, 'R²_Y1']:.4f})")
print(f"Лучшая модель для Y2 (Cooling Load): {results_df.loc[best_y2_idx, 'Model']} (R² =
{results_df.loc[best_y2_idx, 'R²_Y2']:.4f})")

detailed_results = pd.DataFrame({
    'Model': ['lm'] * 2 + ['glm'] * 2 + ['nlm'] * 2 + ['nnet'] * 2,
    'Target': ['Y1', 'Y2'] * 4,
    'R²': [
        r2_score(y_test["Y1"], pred_lm[:, 0]),
        r2_score(y_test["Y2"], pred_lm[:, 1]),
        r2_score(y_test["Y1"], pred_glm[:, 0]),
        r2_score(y_test["Y2"], pred_glm[:, 1]),
        r2_score(y_test["Y1"], pred_nlm_y1),
        r2_score(y_test["Y2"], pred_nlm_y2),
```

```
        r2_score(y_test["Y1"], pred_nn[:, 0]),
        r2_score(y_test["Y2"], pred_nn[:, 1])
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test["Y1"], pred_lm[:, 0])),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_lm[:, 1])),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_glm[:, 0])),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_glm[:, 1])),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_nlm_y1)),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_nlm_y2)),
        np.sqrt(mean_squared_error(y_test["Y1"], pred_nn[:, 0])),
        np.sqrt(mean_squared_error(y_test["Y2"], pred_nn[:, 1]))
    ]
})

print("\nDetailed Results:")
print(detailed_results)

# 6. Для лучшей по точности на тестовом множестве линейной по параметрам модели
# определите значения параметров в разных нормах, в том числе для ошибки MAPE с
# применением медианной регрессии

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy.optimize import minimize

print("=" * 80)
print("6. АНАЛИЗ ПАРАМЕТРОВ ЛУЧШЕЙ МОДЕЛИ (GLM) В РАЗНЫХ НОРМАХ")
print("=" * 80)

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

feature_names = poly.get_feature_names_out(X.columns)

print(f"Количество признаков после полиномиального преобразования: {len(feature_names)}")
print("Первые 10 признаков:", feature_names[:10])

# 6.1 МНК регрессия (L2 норма) - стандартный подход
print("6.1 МНК РЕГРЕССИЯ (L2 НОРМА)")

glm_l2 = LinearRegression()
glm_l2.fit(X_train_poly, y_train)

print("\nПараметры для Y1 (Heating Load):")
params_y1_l2 = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': glm_l2.coef_[0],
    'Abs_Coefficient': np.abs(glm_l2.coef_[0])
}).sort_values('Abs_Coefficient', ascending=False)

print(params_y1_l2.head(10))
print(f"\nIntercept для Y1: {glm_l2.intercept_[0]:.6f}")

print("\nПараметры для Y2 (Cooling Load):")
params_y2_l2 = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': glm_l2.coef_[1],
    'Abs_Coefficient': np.abs(glm_l2.coef_[1])
}).sort_values('Abs_Coefficient', ascending=False)

print(params_y2_l2.head(10))
print(f"\nIntercept для Y2: {glm_l2.intercept_[1]:.6f}")

# 6.2 MAE регрессия (L1 норма) - медианная регрессия
```

```
print("\n6.2 MAE РЕГРЕССИЯ (L1 НОРМА) - МЕДИАННАЯ РЕГРЕССИЯ")

# Функция для L1 регрессии (медианная регрессия)
def l1_regression(X, y):
    def mae_loss(params):
        return np.mean(np.abs(y - X @ params))

    initial_params = np.linalg.lstsq(X, y, rcond=None)[0]

    result = minimize(mae_loss, initial_params, method='BFGS')
    return result.x

def mape_regression(X, y, eps=1e-8):
    # Small epsilon to avoid division by zero
    y_safe = np.where(np.abs(y) < eps, eps, y)

    def mape_loss(params):
        y_pred = X @ params
        return np.mean(np.abs((y_safe - y_pred) / y_pred))

    initial_params = np.linalg.lstsq(X, y, rcond=None)[0]
    result = minimize(mape_loss, initial_params, method='BFGS')
    return result.x

params_y1_l1 = l1_regression(X_train_poly, y_train["Y1"])
params_y2_l1 = l1_regression(X_train_poly, y_train["Y2"])
params_mape_y1 = mape_regression(X_train_poly, y_train["Y1"])
params_mape_y2 = mape_regression(X_train_poly, y_train["Y2"])

print("\nПараметры для Y1 (L1 норма):")
params_y1_l1_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': params_y1_l1,
    'Abs_Coefficient': np.abs(params_y1_l1)
}).sort_values('Abs_Coefficient', ascending=False)

print(params_y1_l1_df.head(10))

print("\nПараметры для Y2 (L1 норма):")
params_y2_l1_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': params_y2_l1,
    'Abs_Coefficient': np.abs(params_y2_l1)
}).sort_values('Abs_Coefficient', ascending=False)

print(params_y2_l1_df.head(10))

print("\nПараметры для Y1 (MAPE):")
params_mape_y1 = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': params_mape_y1,
    'Abs_Coefficient': np.abs(params_mape_y1)
}).sort_values('Abs_Coefficient', ascending=False)
print(params_mape_y1.head(10))

print("\nПараметры для Y2 (MAPE):")
params_mape_y2 = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': params_mape_y2,
    'Abs_Coefficient': np.abs(params_mape_y2)
}).sort_values('Abs_Coefficient', ascending=False)
print(params_mape_y2.head(10))

# 6.4 Сравнение производительности разных норм
print("\n6.4 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ РАЗНЫХ НОРМ")

def evaluate_model_performance_single(y_true, y_pred, model_name, target_name):
    """Оценка производительности модели для одного таргета"""
    mse = mean_squared_error(y_true, y_pred)
```

```
    mae = mean_absolute_error(y_true, y_pred)

    # MAPE
    def mean_absolute_percentage_error(y_true, y_pred):
        return np.mean(np.abs((y_true - y_pred) / y_pred)) * 100

    mape = mean_absolute_percentage_error(y_true, y_pred)

    return {
        'Model': model_name,
        'Target': target_name,
        'MSE': mse,
        'RMSE': np.sqrt(mse),
        'MAE': mae,
        'MAPE': mape
    }

results_y1 = []

y_pred_l2_y1 = glm_l2.predict(X_test_poly)[:, 0]
results_y1.append(evaluate_model_performance_single(y_test["Y1"], y_pred_l2_y1, "L2 (МНК)",
"Y1"))

y_pred_l1_y1 = X_test_poly @ params_y1_l1
results_y1.append(evaluate_model_performance_single(y_test["Y1"], y_pred_l1_y1, "L1 (Медианная)",
"Y1"))

results_y2 = []

y_pred_l2_y2 = glm_l2.predict(X_test_poly)[:, 1]
results_y2.append(evaluate_model_performance_single(y_test["Y2"], y_pred_l2_y2, "L2 (МНК)",
"Y2"))

y_pred_l1_y2 = X_test_poly @ params_y2_l1
results_y2.append(evaluate_model_performance_single(y_test["Y2"], y_pred_l1_y2, "L1 (Медианная)",
"Y2"))

results_df = pd.DataFrame(results_y1 + results_y2)
print("Сравнение производительности:")
print(results_df.round(4))

print("\n6.5 АНАЛИЗ НАИБОЛЕЕ ВАЖНЫХ ПРИЗНАКОВ")

print("Топ-10 признаков для Y1 по разным нормам:")

top_features_comparison = pd.DataFrame({
    'L2_Norm': params_y1_l2.head(10)['Feature'].values,
    'L1_Norm': params_y1_l1_df.head(10)['Feature'].values,
})

print(top_features_comparison)

top_15_features = params_y1_l2.head(15)['Feature'].values

# 6.6 Визуализация сравнения параметров
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(20, 12))
fig.suptitle('Сравнение параметров GLM модели в разных нормах', fontsize=16)

features_idx = range(len(top_15_features))

# Y2 - сравнение коэффициентов
top_15_features_y2 = params_y2_l2.head(15)['Feature'].values

# Находим лучшие модели по MAPE
best_mape_y1 = results_df[results_df['Target'] == 'Y1'].loc[results_df[results_df['Target'] ==
'Y1']['MAPE'].idxmin()]
```

```
best_mape_y2 = results_df[results_df['Target'] == 'Y2'].loc[results_df[results_df['Target'] ==
'Y2']['MAPE'].idxmin()]

print(f"\nЛучшая модель по MAPE для Y1: {best_mape_y1['Model']} (MAPE =
{best_mape_y1['MAPE']:.2f}%)")
print(f"Лучшая модель по MAPE для Y2: {best_mape_y2['Model']} (MAPE = {best_mape_y2['MAPE']:.2f}
%)")

print("\nТоп-5 наиболее важных признаков для энергоэффективности:")
print("Y1 (Heating Load):")
for i, feature in enumerate(top_15_features[:5], 1):
    print(f"  {i}. {feature}")

print("\nY2 (Cooling Load):")
for i, feature in enumerate(top_15_features_y2[:5], 1):
    print(f"  {i}. {feature}")
```